Recora: Recurring Payments with Escrow and Dispute Resolution on Solana

Capstone Deliverable 2: Architecture Diagram & Requirements (group document)
Team: Utkarsh Jaiswal, Akshat Odiya

1. Executive Summary and Actors

1.1 Problem Statement

Recora is a non-custodial recurring pull-payment and escrow protocol on Solana. A consumer subscribes to a service in USDC, and every charge is held in escrow for a review window (`review_secs`) before the merchant is paid. Inside that window the subscriber can dispute the charge, and a resolver named in the plan can settle the dispute.

The design merges the two individual LOIs: SubStack (pull-based subscriptions through a delegated allowance) and Recourse (a gap between authorising a payment and settling it, with refund and dispute paths).

1.2 Vault Model

Every subscription owns its own escrow vault: a PDA token account with seeds `[b"vault", subscription]` whose authority is the Subscription PDA. It is not an Associated Token Account and it is not shared between subscriptions, so there is no single pool holding everyone's funds.

1.3 Token Delegation

The subscriber gives a standard SPL Token approval to one shared program PDA, `[b"delegate"]`. An SPL token account can hold only one delegate, so one shared delegate lets a subscriber have several subscriptions at once. The program, not the allowance, limits each subscription to one `price` per `interval_secs`, and it only pulls from the account that belongs to that subscription's subscriber.

1.4 Keeper Model

Keepers are permissionless callers of `process_billing` and `settle_charge`. In the POC they are unpaid: no tip is taken from payments. The caller deposits the rent for the Charge PDA during billing, and that same rent is returned to them when the charge is closed. The expected keeper is the merchant, who wants to be paid, but anyone can run both instructions.

1.5 Actors

- **Direct actors (signers)**
  - **Subscriber:** signs `subscribe`, `dispute_charge`, `cancel_subscription` and `close_subscription`.
  - **Merchant:** signs `initialize_merchant`, `create_plan` and `refund_charge`.
  - **Keeper (anyone):** signs `process_billing` and `settle_charge`. It has no authority; the result is the same for any signer.

    - **Resolver:** signs `resolve_dispute`. The resolver is named by the merchant in the plan and is visible to the subscriber before subscribing. It is separate from the protocol admin.
- **Beneficiaries**
    - **Merchant payout ATA:** receives net revenue when a charge settles.
    - **Subscriber USDC ATA:** receives refunds, in full or in part.
    - **Resolver USDC ATA:** receives the resolver fee when a dispute is resolved.
    - **Fee treasury PDA:** receives the protocol fee (at most 100 bps) when a charge settles.
- **Administrators**
    - **Protocol admin:** manages global configuration and fee rates, and withdraws collected fees.
    - **Program upgrade authority:** the deployer key during the POC, moving to a timelocked Squads multisig before mainnet.
    - Administrators cannot access escrow vaults, settle charges or decide disputes.
- **Stakeholders:** off-chain indexers, SDK integration partners, Circle (USDC freeze authority), candidate arbitration services, auditors, and agent frameworks (roadmap).

1.6 Core MVP Use Cases

Template: 1 use case = 1 atomic state transition = 1 Anchor instruction handler.

UC01: Authorize and initialize a subscription

- **Handler:** `subscribe`
- **Signer:** Subscriber
- **State transition:** nothing → Subscription `Active`. Creates the Subscription PDA `[b"sub", plan, subscriber]` and its vault, snapshots the plan terms (price, interval, review window, resolver, fee rates), approves the shared Delegate PDA, and sets `next_billing_ts = now`. It does not transfer tokens; cycle 0 is pulled by `process_billing`.

UC02: Execute a recurring escrow billing cycle

- **Handler:** `process_billing`
- **Signer:** Keeper (anyone)
- **State transition:** nothing → Charge `Held`, or Subscription `Active` → `Lapsed`. Requires `status == Active` and `now ≥ next_billing_ts`, and that `subscriber_ata` belongs to the subscription's subscriber. If balance and allowance cover the price, it pulls the price through the Delegate PDA into the vault, creates the Charge PDA in `Held`, and advances `next_billing_ts` by `interval_secs`. If they do not, it skips the transfer and sets the subscription to `Lapsed`.

UC03: Settle a matured escrow charge

- **Handler:** `settle_charge`
- **Signer:** Keeper (anyone)
- **State transition:** Charge `Held` → `Settled`. Requires `now ≥ pulled_at + review_secs`. Transfers `price − protocol_fee` to the merchant payout ATA and `protocol_fee` to the fee treasury, decrements `open_charges`, closes the Charge PDA and returns its rent to the original `rent_payer`.

The remaining handlers (setup, dispute, refund, resolve, cancel, close) support these three and are listed in the requirements matrix.

2. Granularity Self-Check and Adversarial Red-Teaming

2.1 Granularity Self-Check

- **Rule 1, Atomicity.** Our first version of `subscribe` created the subscription and also took the first payment. That was two transitions in one handler, so we split it: `subscribe` only sets up accounts and the delegation, and `process_billing` handles every payment, including cycle 0. Two bends remain and we declare them: `subscribe` creates the accounts and approves the delegate in one handler (REQ-06, REQ-07), and `settle_charge` has two entry states (REQ-11, REQ-12).
- **Rule 2, State ownership.** Every state change lives in a program-owned account. We rejected a growing list of charges inside the Subscription account. Each cycle gets its own Charge PDA `[b"charge", subscription, cycle]`, and each subscription owns one vault `[b"vault", subscription]`.
- **Rule 3, Real signers.** No backend relayer is assumed. Subscribers sign their own lifecycle instructions, the resolver signs rulings, and the billing and settlement cranks can be signed by anyone.
- **Rule 4, On-chain vs client-side.** Custody, the state machines, deadline checks and fee maths are on-chain. Dispute evidence is stored off-chain and only its 32-byte sha256 hash (`evidence_ref`) goes on-chain. Countdown timers, invoices and alerts are computed by the client from on-chain timestamps.

2.2 Adversarial Red-Teaming

- **What the AI caught that we missed.** If a pull is attempted with insufficient funds, the CPI fails and the whole transaction reverts, so the program can never record that the subscription is delinquent. We fixed this by checking balance and allowance before the CPI and writing `status = Lapsed` on failure, so the transaction succeeds.
- **What the AI caught that we missed.** Because one Delegate PDA is approved on every subscriber's account, `process_billing` must check that the account being charged belongs to that subscription's subscriber. Without this check, a caller could bill one subscription from a different subscriber's wallet.
- **What the AI flagged that we overrode.** The AI suggested an internal timer hook so billing would run without an external caller. We rejected it. A Solana program cannot invoke itself or pay for its own transaction; someone has to submit and sign it. Billing therefore stays a permissionless instruction that a keeper calls once `next_billing_ts` has passed.

3. On-Chain Requirements Matrix

Req ID	Atomic functional statement	Anchor handler	Required accounts and PDAs	CPI dependencies	Custom constraints and error paths
REQ-01	Initialize the global protocol configuration and the fee treasury.	`initialize_config`	• config: PDA [b"config"] (init) • treasury: PDA token account [b"treasury"] (init, authority = config) • mint: USDC • program, program_data • deployer: Signer	System Program; SPL Token Program (initialize_account)	Deployer must equal the program upgrade authority (NotUpgradeAuthority). fee_bps ≤ 100 and max_resolver_fee_bps ≤ 1000 (FeeTooHigh). min ≤ max for every window bound (InvalidWindowBounds).
REQ-02	Update the global configuration: fee, pause flag, window bounds, admin authority.	`update_config`	• config: PDA [b"config"] (mut) • admin: Signer	None	Signer must equal config.authority (Unauthorized). fee_bps ≤ 100. Two-step authority handover (propose, accept). Pause blocks create_plan and subscribe only; live subscriptions and charges are never affected.
REQ-03	Withdraw accumulated protocol fees from the fee treasury.	`withdraw_fees`	• config: PDA [b"config"] • treasury: PDA [b"treasury"] (mut) • destination: token account (mut) • admin: Signer	SPL Token Program (transfer_checked)	Signer must equal config.authority (Unauthorized). amount ≤ treasury.amount (InsufficientFees). Config PDA signs the transfer. No subscription vault is reachable.
REQ-04	Register a merchant profile and its payout account.	`initialize_merchant`	• merchant: PDA [b"merchant", merchant_authority] (init) • payout_ata: merchant USDC ATA • merchant_authority: Signer	System Program	payout_ata must be the canonical USDC ATA of the merchant wallet (WrongTokenAccount). One profile per merchant authority.
REQ-05	Create a subscription plan with price, billing interval, review window and resolver.	`create_plan`	• plan: PDA [b"plan", merchant_authority, plan_id] (init) • merchant: PDA (mut) • config: PDA • merchant_authority: Signer	System Program	Protocol not paused (ProtocolPaused). price > 0, min_interval ≤ interval_secs ≤ max_interval, min_review ≤ review_secs ≤ max_review (InvalidTerms). resolver ≠ merchant and resolver_fee_bps ≤ config cap (InvalidResolver).
REQ-06	Initialize the subscription record and its escrow vault, without moving funds.	`subscribe`	• sub: PDA [b"sub", plan, subscriber] (init) • vault: PDA token account [b"vault", sub] (init, authority = sub) • plan: PDA (mut) • config: PDA • subscriber: Signer	System Program; SPL Token Program (initialize_account)	Protocol not paused (ProtocolPaused). subscriber ≠ merchant and subscriber ≠ resolver (PartyConflict). Snapshots price, fee, interval, review window, dispute timeout and resolver. Sets status = Active and next_billing_ts = now. Does not transfer tokens.

REQ-07	Approve the shared Delegate PDA on the subscriber's USDC account.	`subscribe`	• subscriber_ata: subscriber USDC ATA (mut) • delegate: PDA [b"delegate"] • subscriber: Signer	SPL Token Program (approve)	subscriber_ata must be the canonical USDC ATA of the subscriber (WrongTokenAccount). Allowance = price × n_cycles, added to the existing allowance if the delegate is already the Delegate PDA.
REQ-08	Pull one cycle's price into the subscription's escrow vault and record it as a Held charge.	`process_billing`	• sub: PDA (mut) • charge: PDA [b"charge", sub, cycle] (init) • subscriber_ata (mut) • vault: PDA [b"vault", sub] (mut) • delegate: PDA [b"delegate"] • caller: Signer (pays Charge rent)	System Program; SPL Token Program (transfer_checked)	status == Active (SubscriptionNotActive). now ≥ next_billing_ts (NotDue). subscriber_ata must be the canonical USDC ATA of sub.subscriber (WrongTokenAccount). Delegate PDA signs. Charge state = Held, next_billing_ts += interval_secs.
REQ-09	Move the subscription to Lapsed when the pull cannot be funded.	`process_billing`	• sub: PDA (mut) • plan: PDA (mut) • subscriber_ata • caller: Signer	None	Checked before the CPI: delegate == Delegate PDA, delegated_amount ≥ price, balance ≥ price. If any fails, status = Lapsed, no transfer is attempted and the transaction succeeds.
REQ-10	Let the subscriber dispute a Held charge inside the review window.	`dispute_charge`	• charge: PDA [b"charge", sub, cycle] (mut) • sub: PDA • subscriber: Signer	None	subscriber == sub.subscriber (Unauthorized). state == Held (InvalidState). now < pulled_at + review_secs (ReviewWindowElapsed). Stores evidence_ref and sets state = Disputed.
REQ-11	Settle a Held charge to the merchant and the fee treasury after the review window.	`settle_charge`	• charge: PDA (mut, close) • sub: PDA (mut) • vault: PDA (mut) • merchant: PDA (mut) • merchant_ata (mut) • treasury: PDA (mut) • config: PDA • rent_payer (mut) • caller: Signer	SPL Token Program (transfer_checked)	state == Held and now ≥ pulled_at + review_secs (ReviewWindowNotElapsed). Destinations must match stored keys (WrongAccount). price − fee to the merchant, fee to the treasury. Closes the Charge and returns rent to rent_payer.
REQ-12	Refund the subscriber when a dispute times out with no ruling.	`settle_charge`	• charge: PDA (mut, close) • sub: PDA (mut) • vault: PDA (mut) • subscriber_ata (mut) • rent_payer (mut) • caller: Signer	SPL Token Program (transfer_checked)	state == Disputed and now ≥ dispute_opened_at + dispute_secs (DisputeNotTimedOut). Refunds 100% to the subscriber with no protocol fee. Closes the Charge and returns rent to rent_payer.

REQ-13	Let the merchant refund a Held or Disputed charge in full.	`refund_charge`	• charge: PDA (mut, close) • sub: PDA (mut) • vault: PDA (mut) • subscriber_ata (mut) • rent_payer (mut) • merchant_authority: Signer	SPL Token Program (transfer_checked)	Signer must equal the merchant authority (Unauthorized). state in {Held, Disputed} (InvalidState). Transfers 100% back to the subscriber with no fee. Closes the Charge.
REQ-14	Let the plan's resolver split a Disputed charge between the parties.	`resolve_dispute`	• charge: PDA (mut, close) • sub: PDA (mut) • vault: PDA (mut) • subscriber_ata, merchant_ata, resolver_ata (mut) • treasury: PDA (mut) • rent_payer (mut) • resolver: Signer	SPL Token Program (transfer_checked)	resolver == sub.resolver snapshot (Unauthorized). state == Disputed (InvalidState). merchant_bps ≤ 10 000 (InvalidSplit). Up to four transfers (subscriber, merchant, resolver, treasury); the parts must sum to the charge amount. Closes the Charge.
REQ-15	Cancel a subscription so that no future cycle is billed.	`cancel_subscription`	• sub: PDA (mut) • plan: PDA (mut) • subscriber: Signer	None	subscriber == sub.subscriber (Unauthorized). status in {Active, Lapsed} (InvalidState). Sets status = Cancelled. Charges already Held still run to their own terminal state.
REQ-16	Close a finished subscription and its vault and return the rent.	`close_subscription`	• sub: PDA (mut, close) • vault: PDA (mut, close) • subscriber_ata (mut) • subscriber: Signer	SPL Token Program (transfer_checked, close_account)	subscriber == sub.subscriber (Unauthorized). status in {Cancelled, Lapsed} (StillActive). open_charges == 0 (ChargesOutstanding). Sweeps any residual dust to the subscriber, then returns rent to the subscriber.

4. Architecture Diagrams

The diagrams follow one structure: a legend, an overview, the account model, one flow per instruction with every decision and error path, and an end-to-end view with external actors. REQ labels on the diagrams match the matrix in Section 3.

Figure 1: Architecture Notation and Legend

Node shapes, colours and arrow styles used in every diagram.

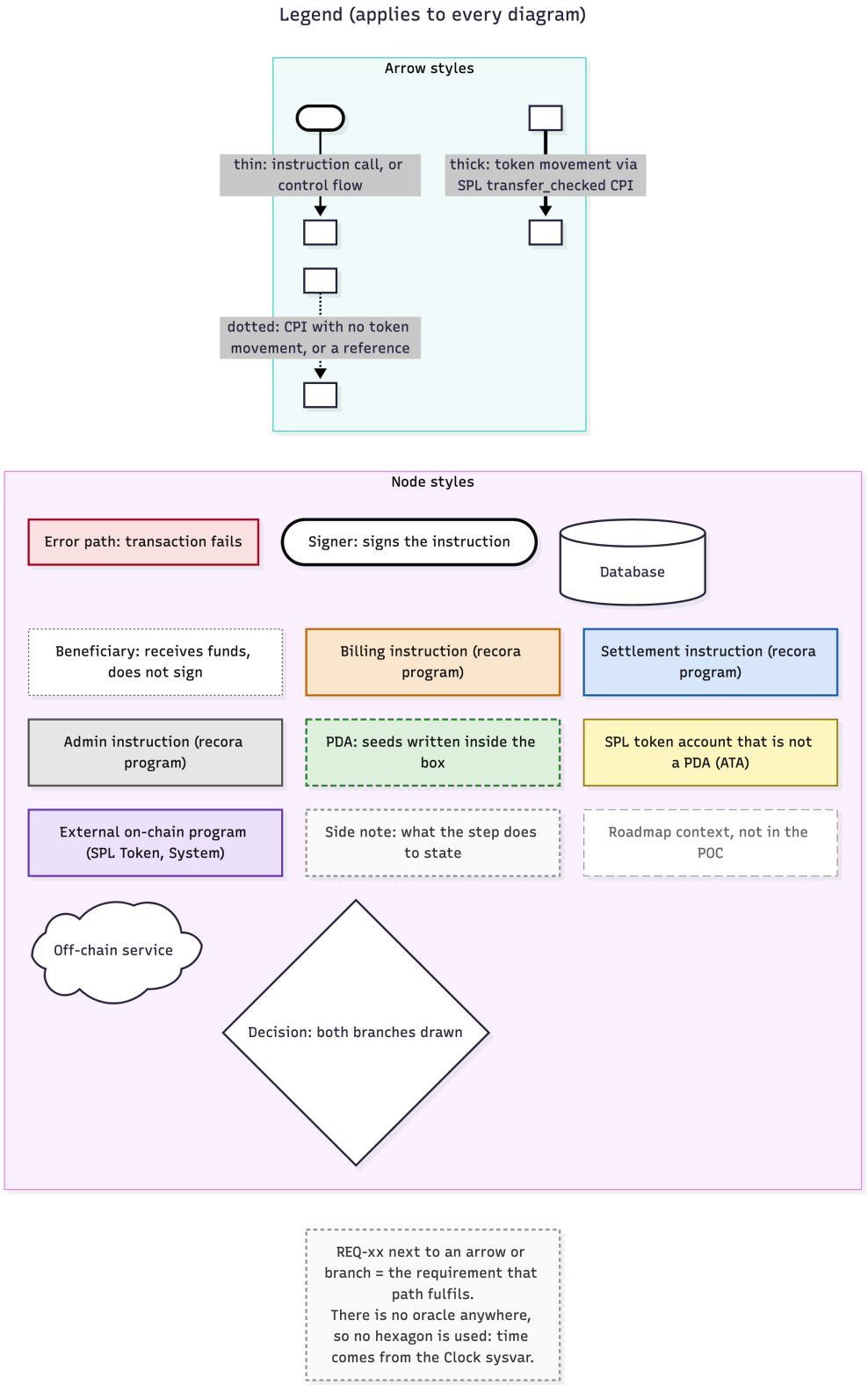


Figure 2: Recora High-Level Architecture Overview

The whole protocol on one page. Each numbered arrow is one instruction, labelled with the requirement it fulfils.

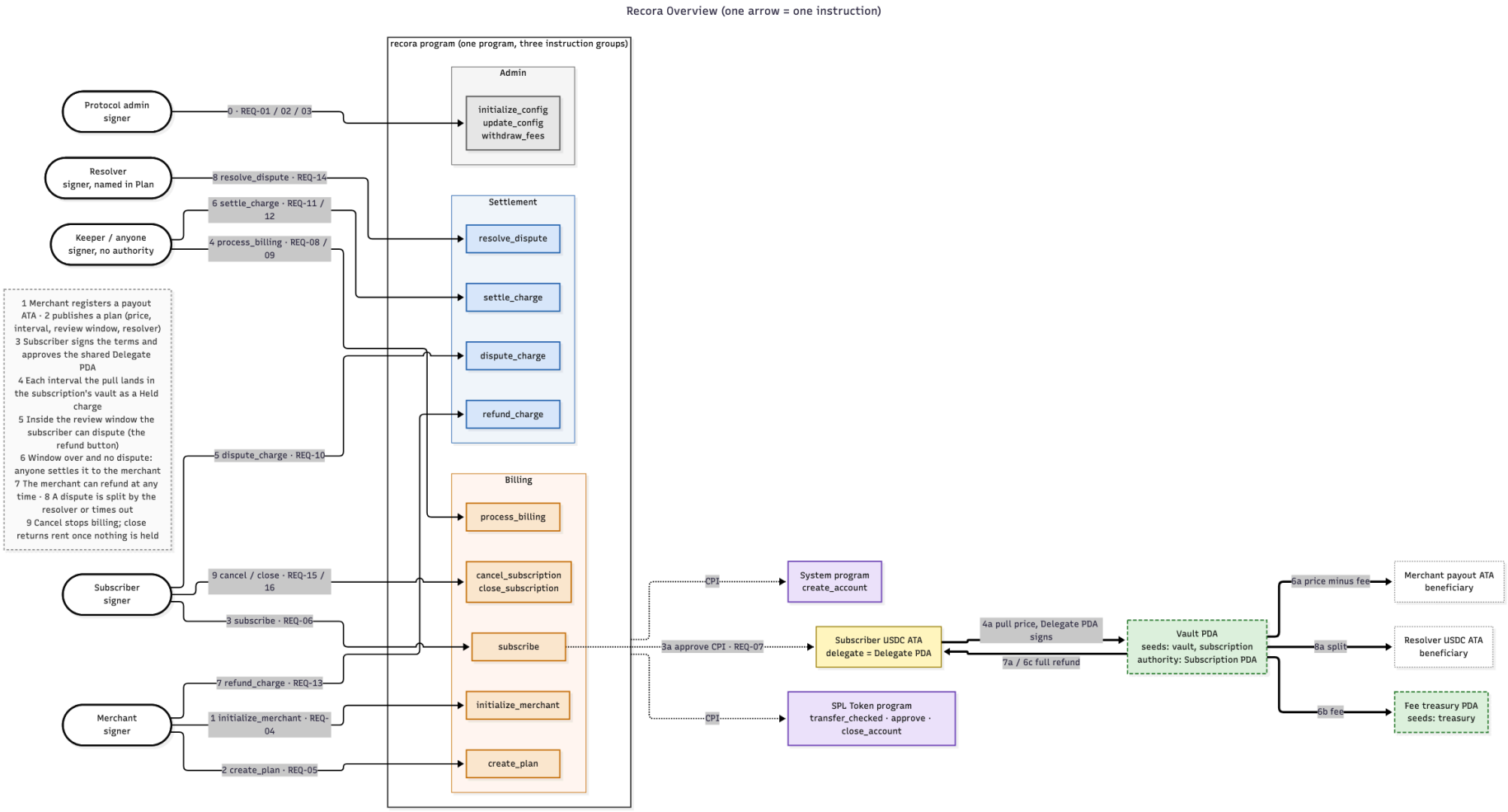


Figure 3: Actor Categories: Direct Actors, Beneficiaries, Administrators, Stakeholders

Which actor signs which instruction, and which accounts only receive funds.

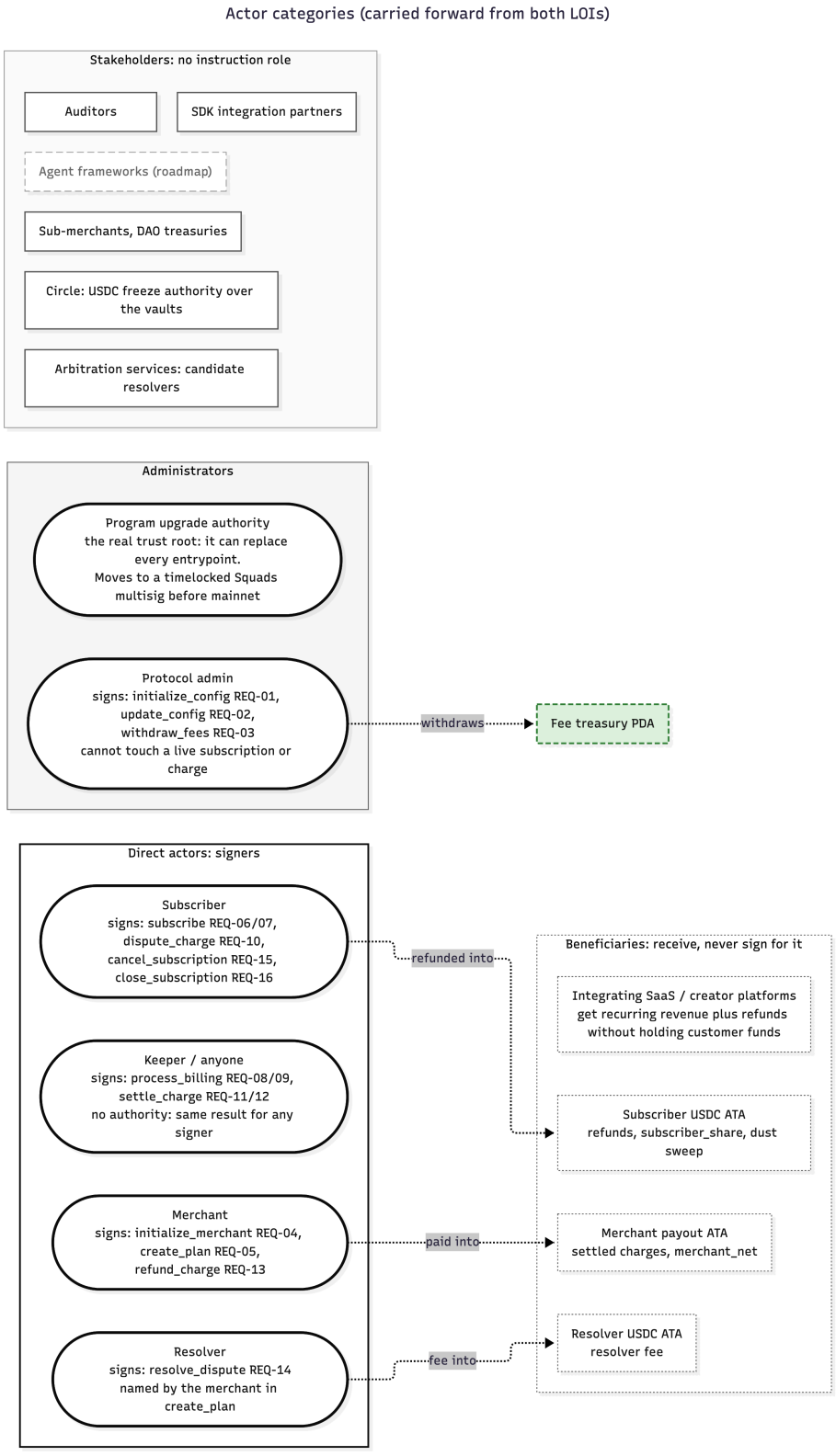


Figure 4: Program Accounts, PDA Seeds and Escrow Vault Layout

Every account with its seeds, owning program, authority and stored fields.

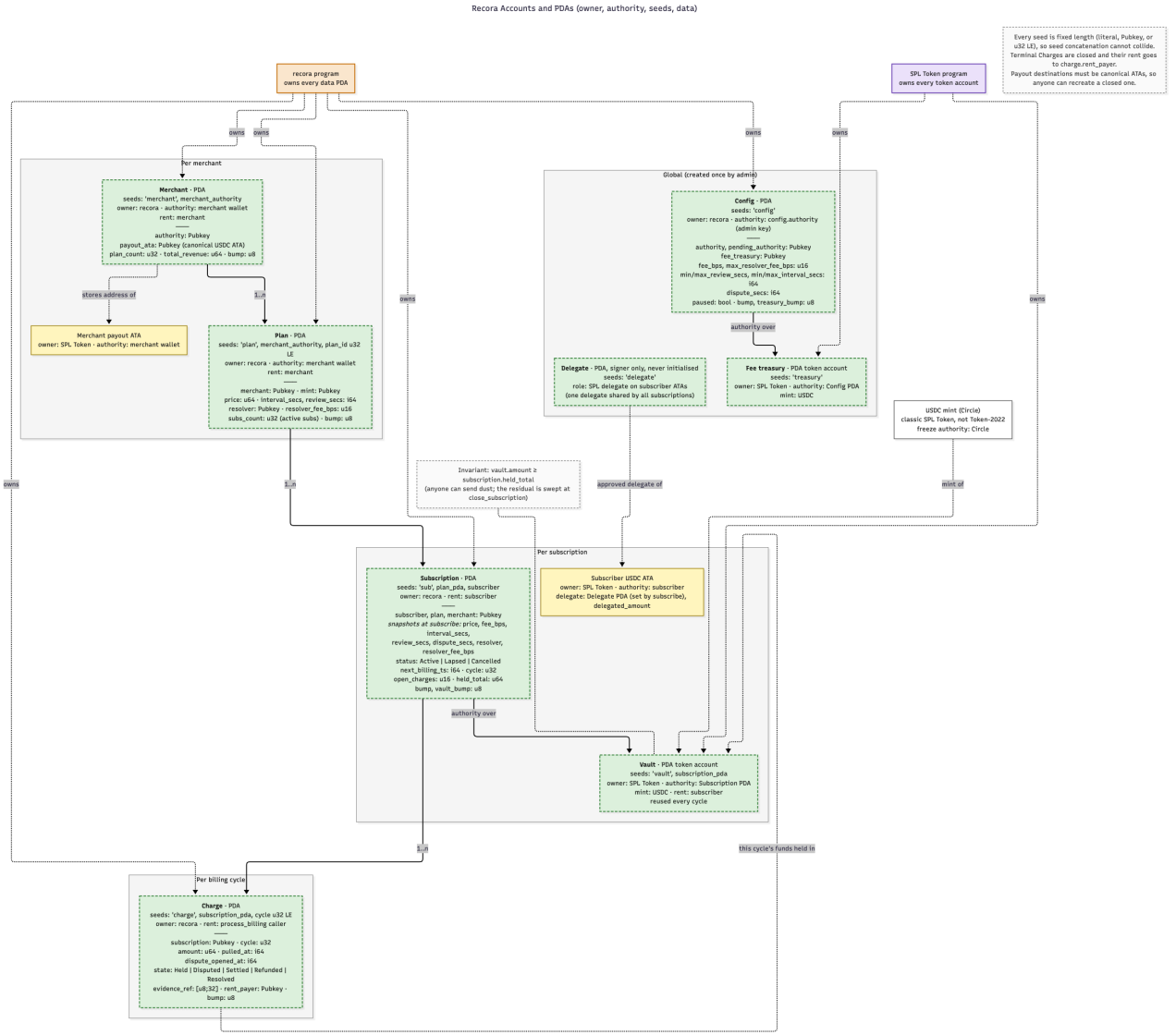


Figure 5: Subscription and Charge State Machines

Subscription states (Active, Lapsed, Cancelled) and Charge states (Held, Disputed, Settled, Refunded, Resolved).

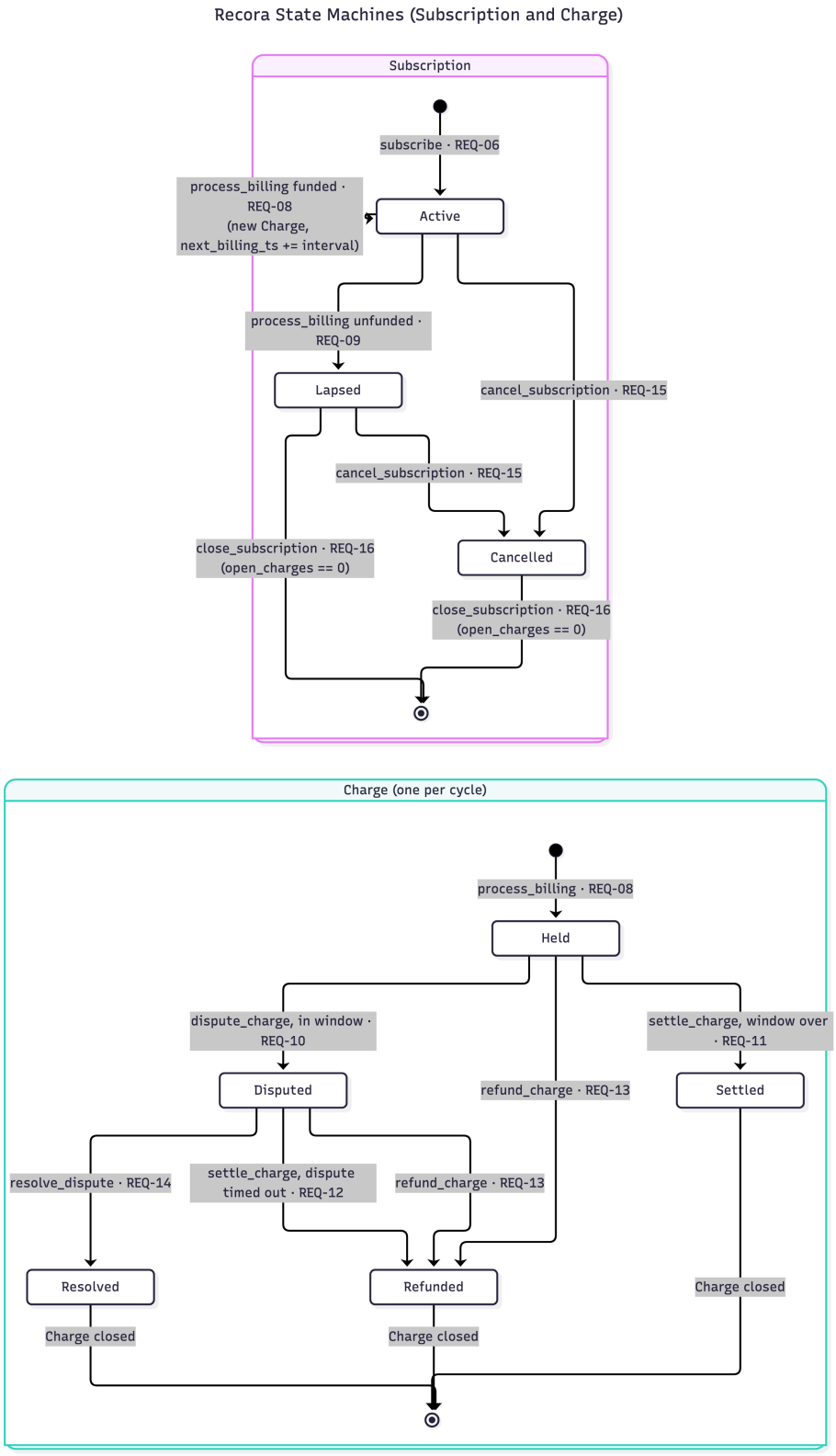


Figure 6: Instruction Flow: initialize_config [REQ-01]

Creates the Config and the fee treasury. Only the program upgrade authority can run it.

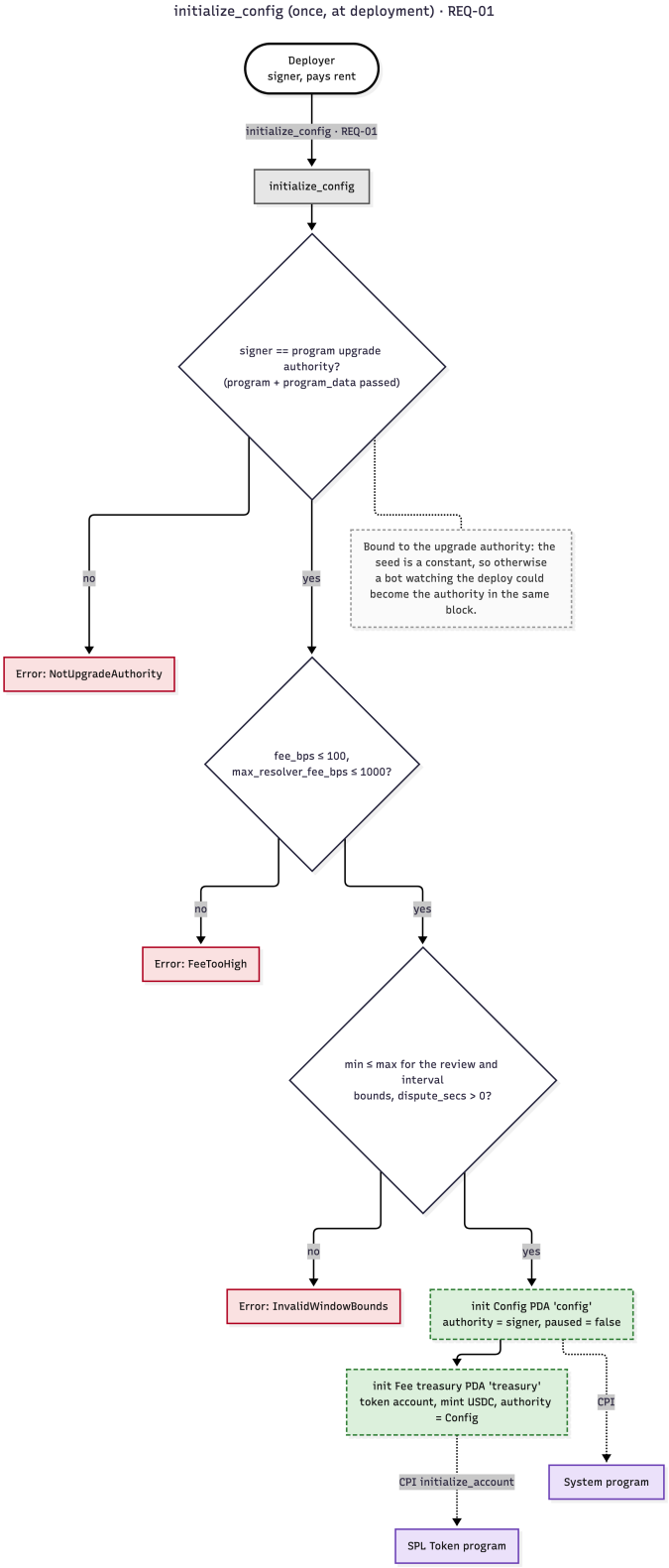


Figure 7: Instruction Flow: update_config [REQ-02]

Five single-purpose admin instructions. None of them can change a live subscription or charge.

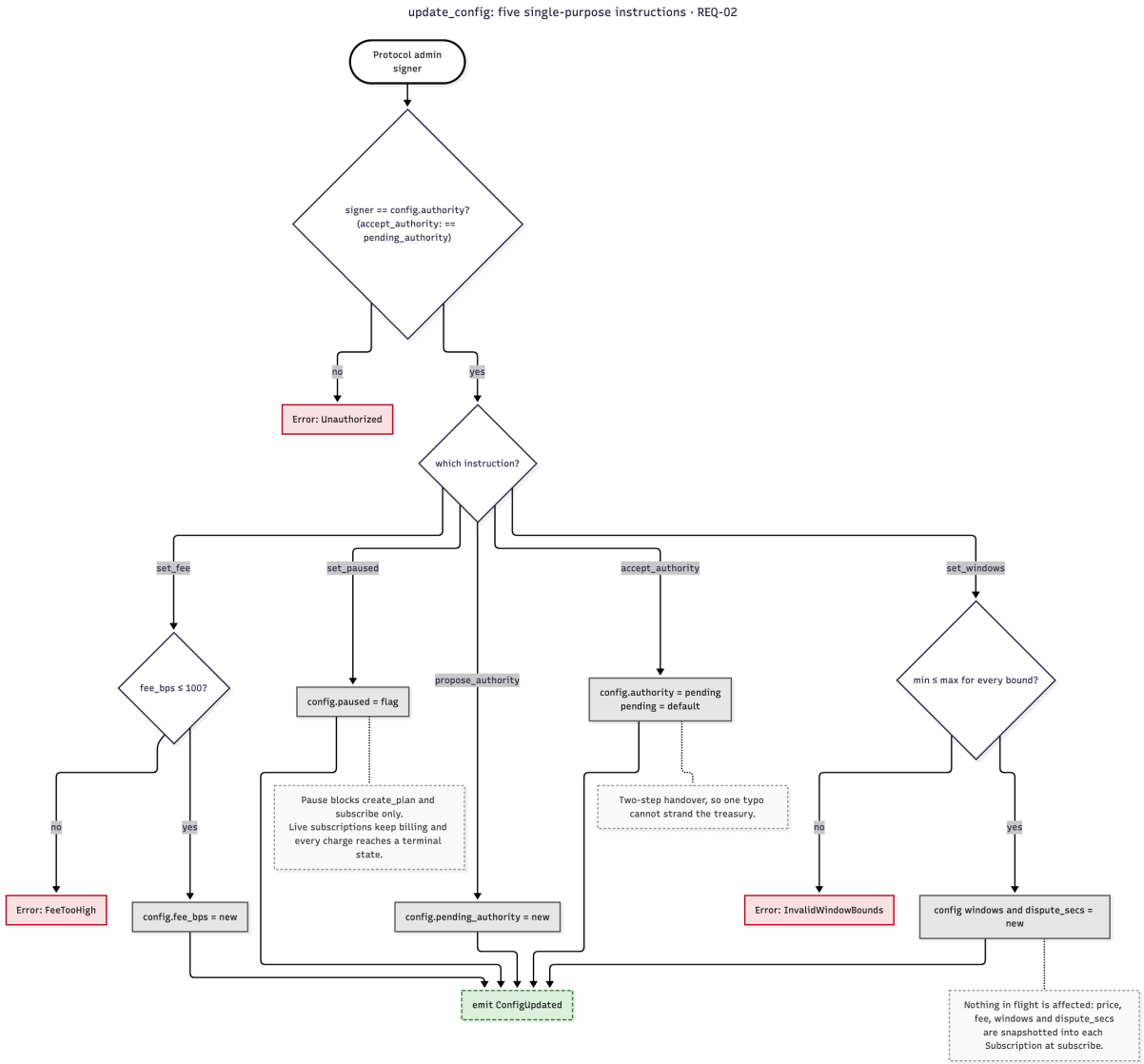


Figure 8: Instruction Flow: withdraw_fees [REQ-03]

The admin withdraws collected protocol fees from the fee treasury.

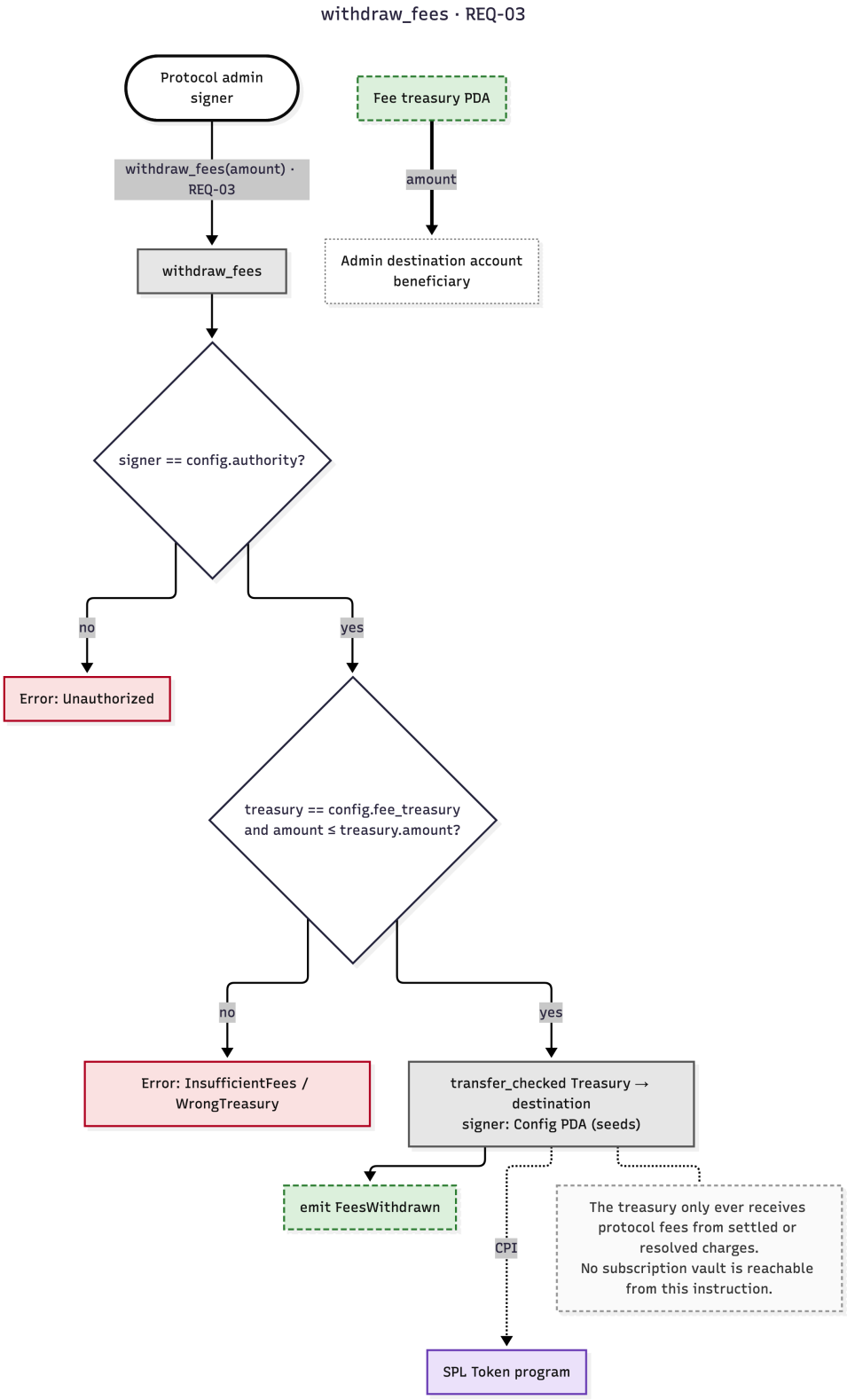


Figure 9: Instruction Flow: initialize_merchant [REQ-04]

A merchant registers a profile and a canonical USDC payout account.

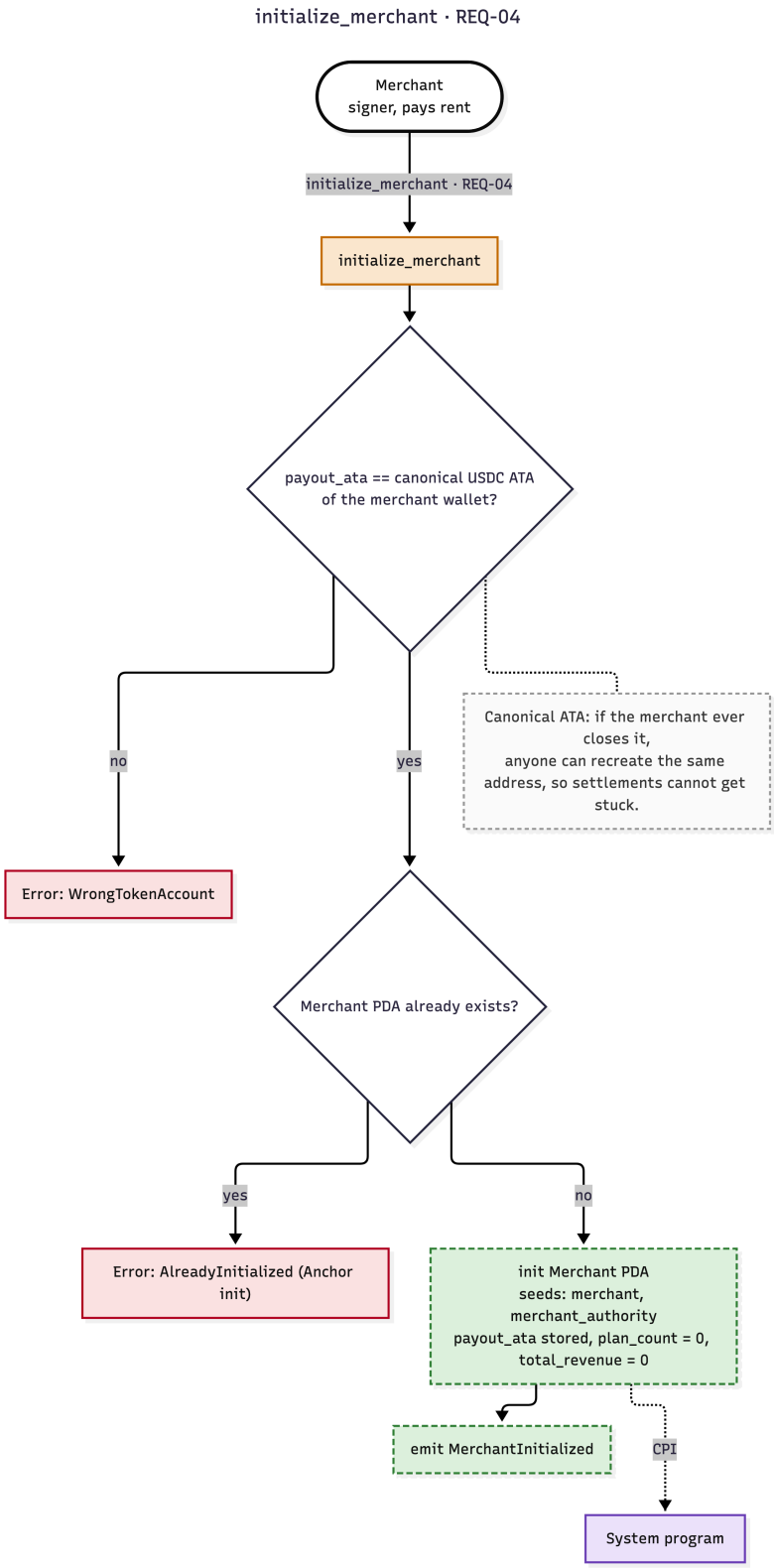


Figure 10: Instruction Flow: create_plan [REQ-05]

A merchant publishes a plan: price, interval, review window and resolver, all checked against Config bounds.

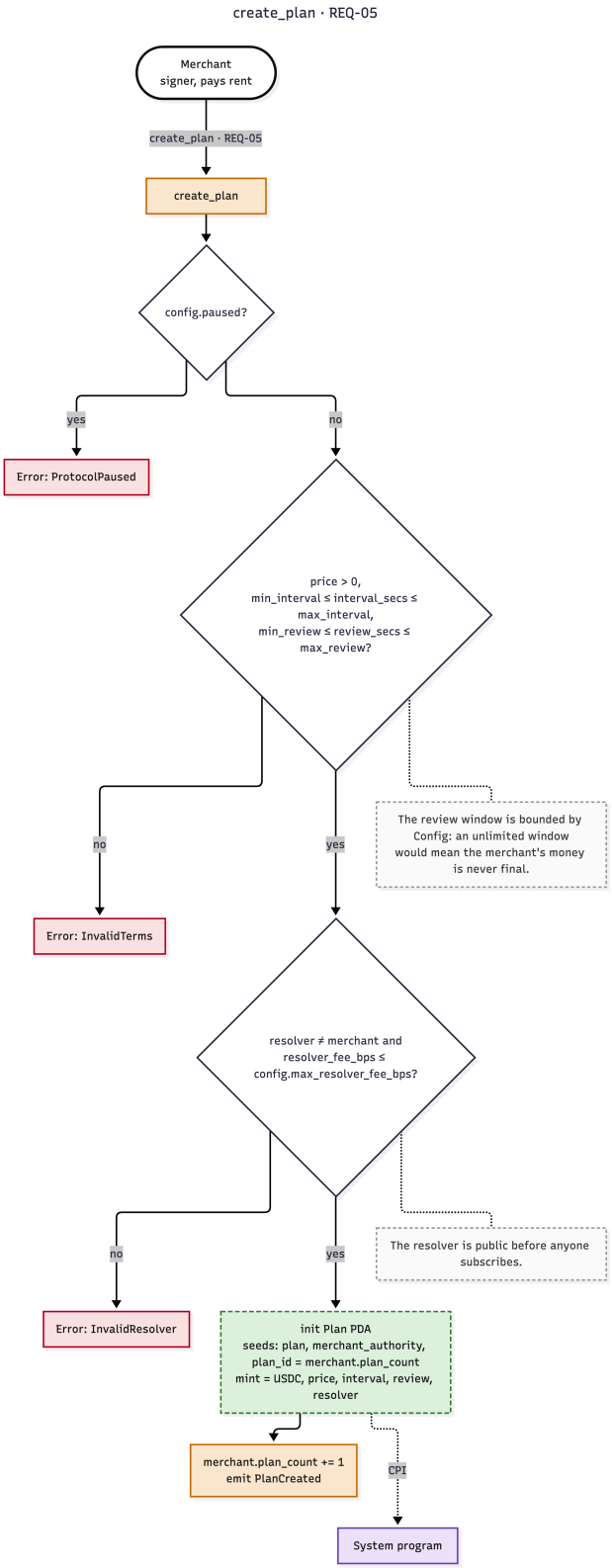


Figure 11: Instruction Flow: subscribe [REQ-06, REQ-07]

The subscriber signs the plan terms. The Subscription and its vault are created and the shared Delegate PDA is approved. No funds move.

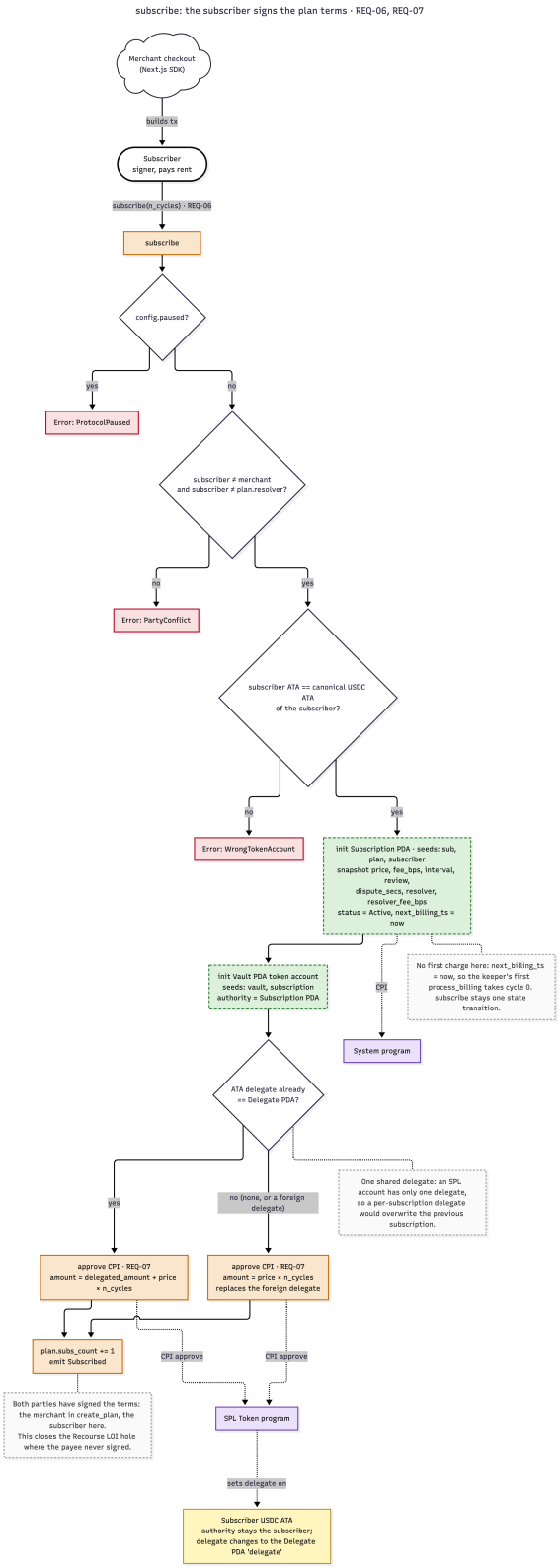


Figure 12: Instruction Flow: process_billing [REQ-08, REQ-09]

Anyone can run it once a cycle is due. A funded pull lands in the vault as a Held charge. An unfunded one moves the subscription to Lapsed without reverting.

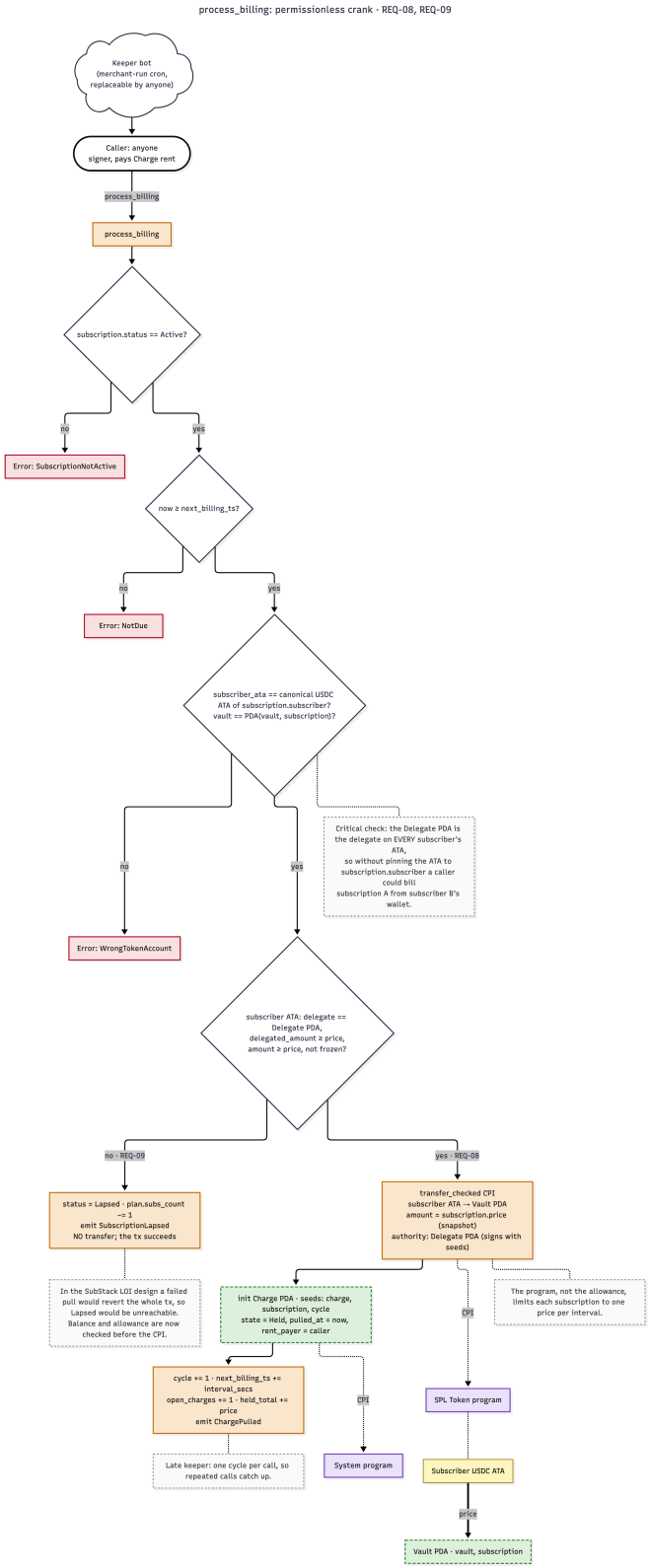


Figure 13: Instruction Flow: cancel_subscription [REQ-15]

The subscriber stops future billing. Charges already held are not affected.

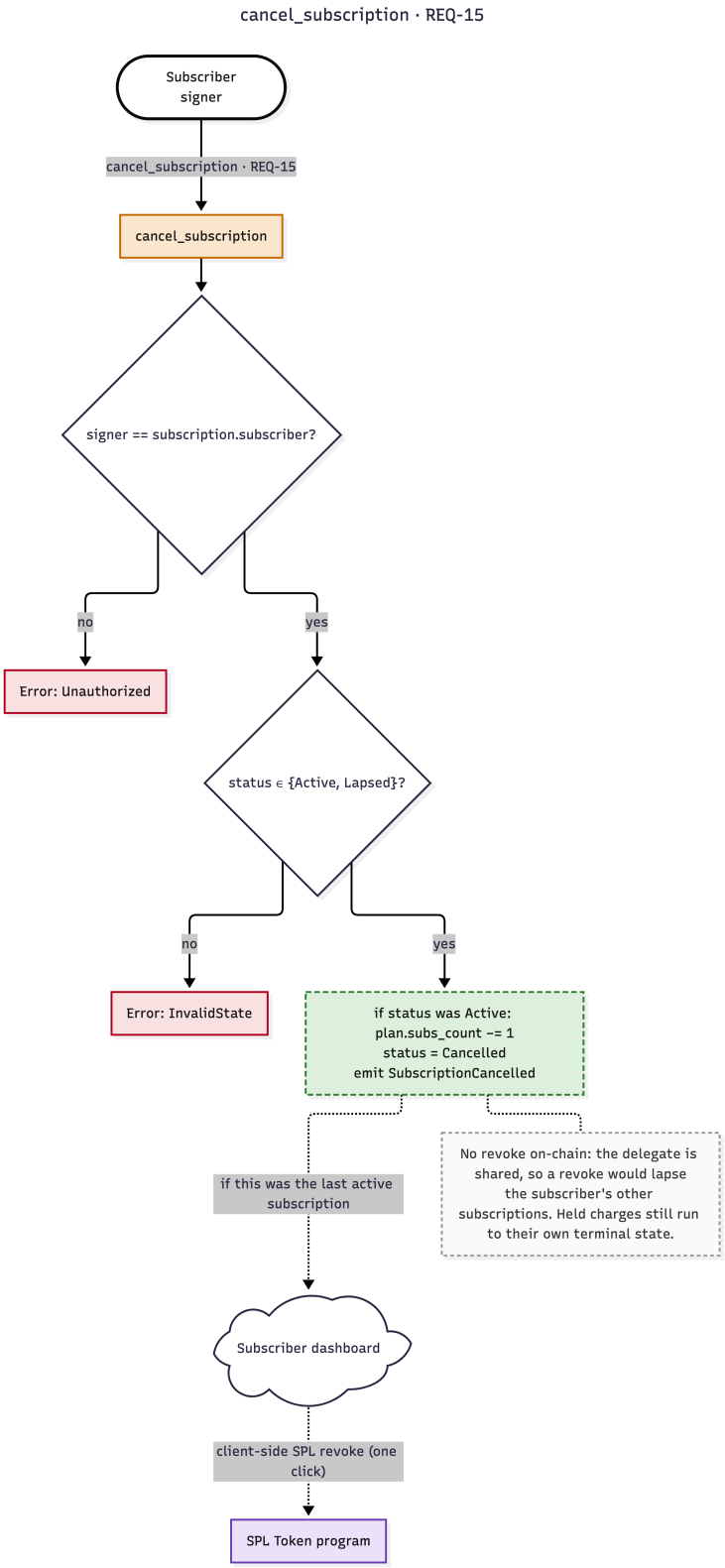


Figure 14: Instruction Flow: close_subscription [REQ-16]

Once nothing is held, the vault and Subscription are closed and the rent goes back to the subscriber.

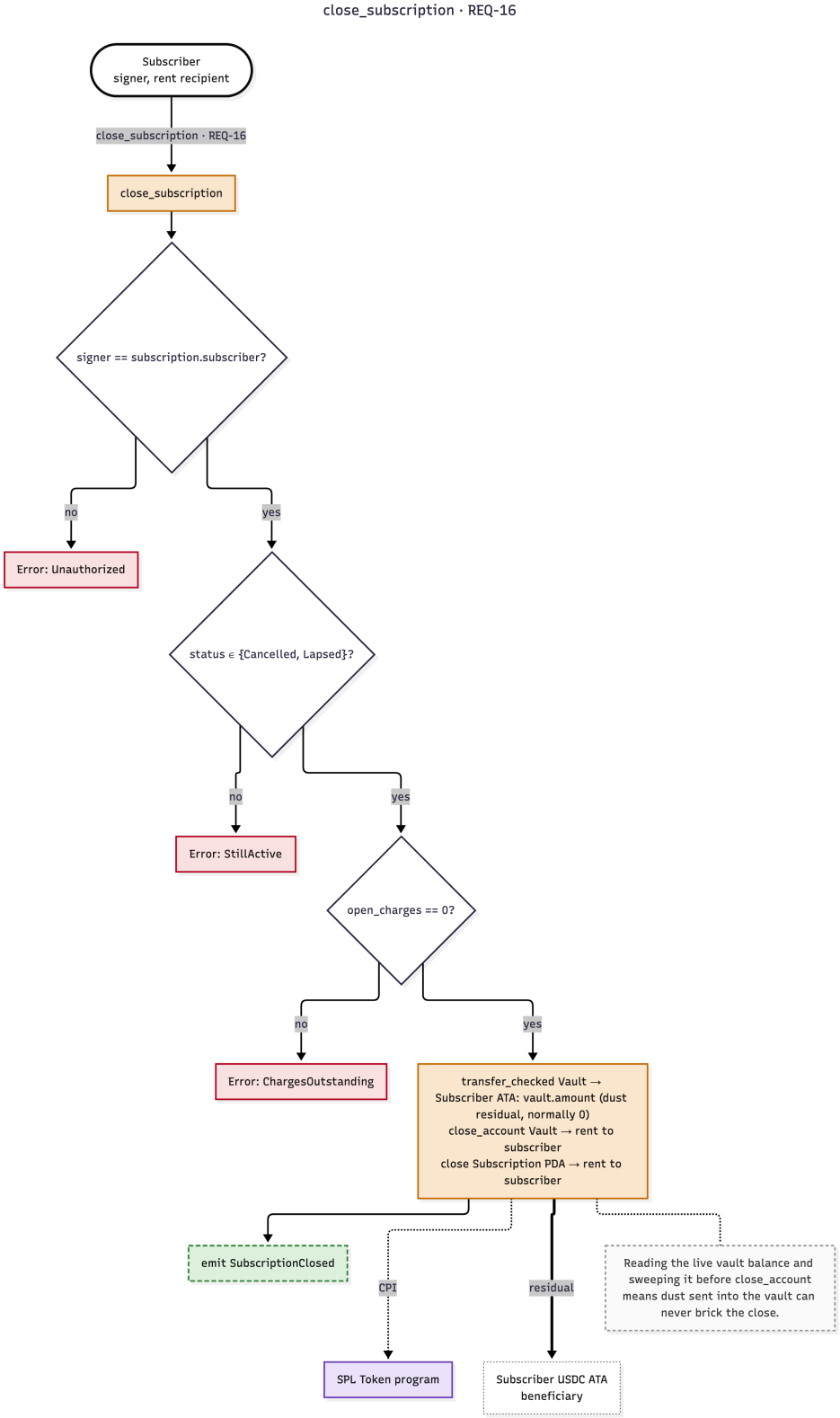


Figure 15: Instruction Flow: dispute_charge [REQ-10]

Inside the review window the subscriber can dispute a Held charge. Only the hash of the evidence goes on-chain.

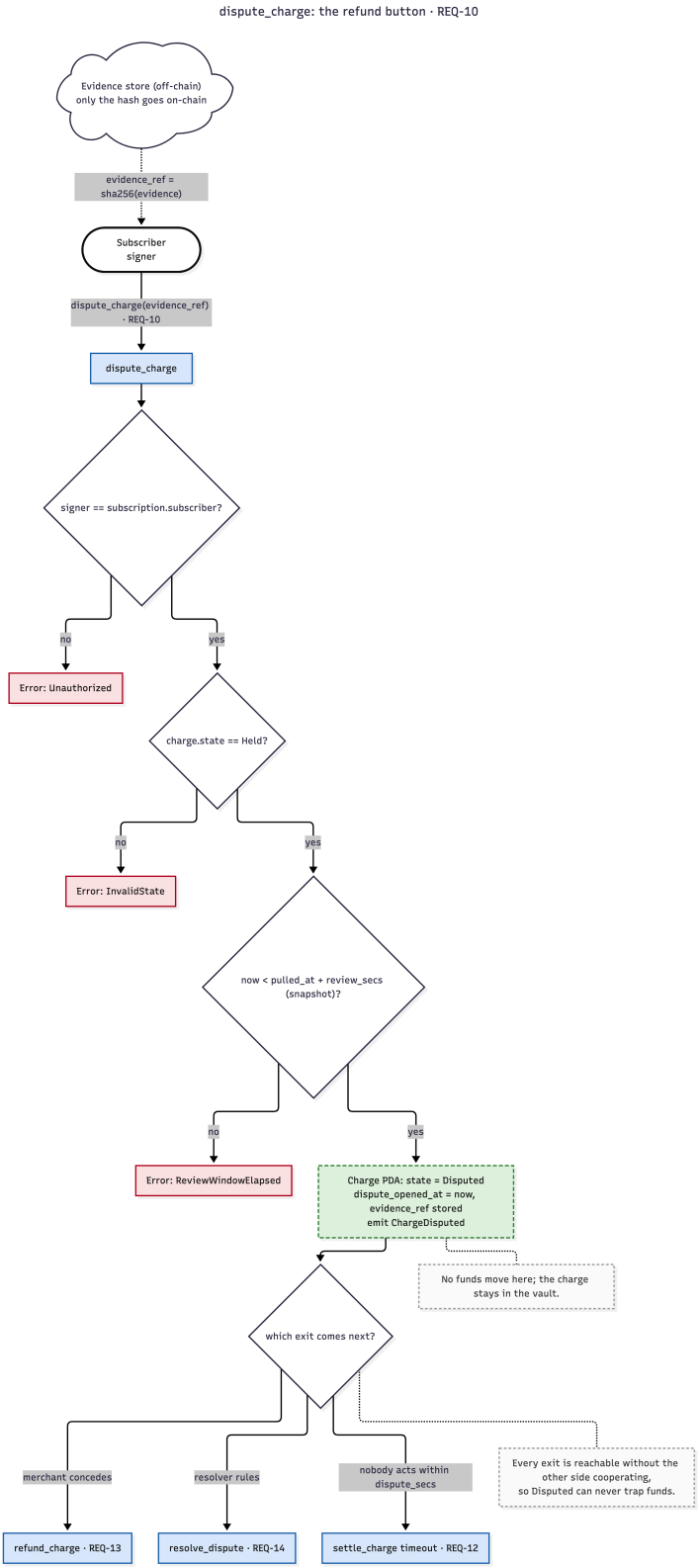


Figure 16: Instruction Flow: settle_charge [REQ-11, REQ-12]

After the review window a Held charge settles to the merchant. A dispute that times out refunds the subscriber.

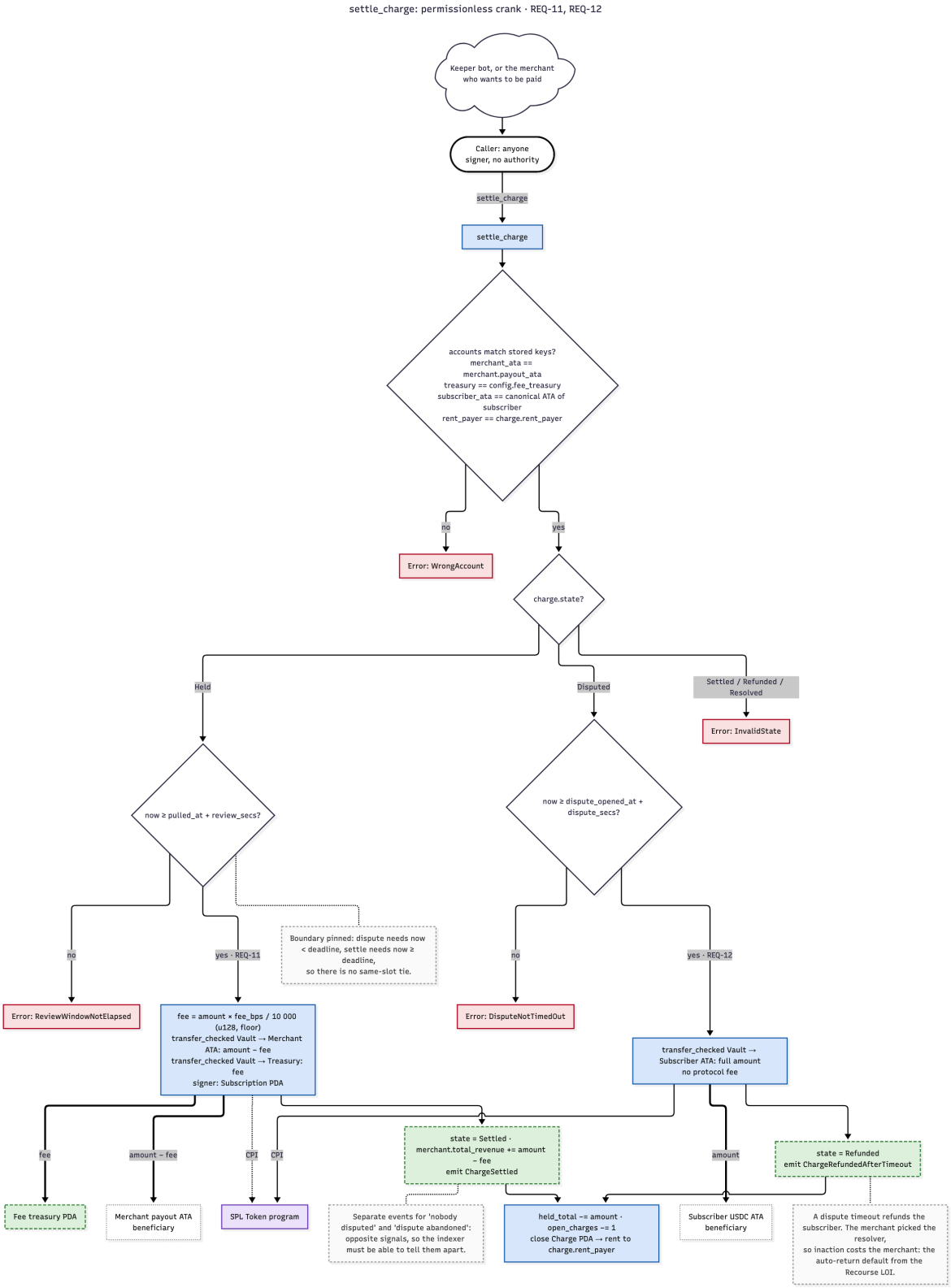


Figure 17: Instruction Flow: refund_charge [REQ-13]

The merchant returns a Held or Disputed charge in full.

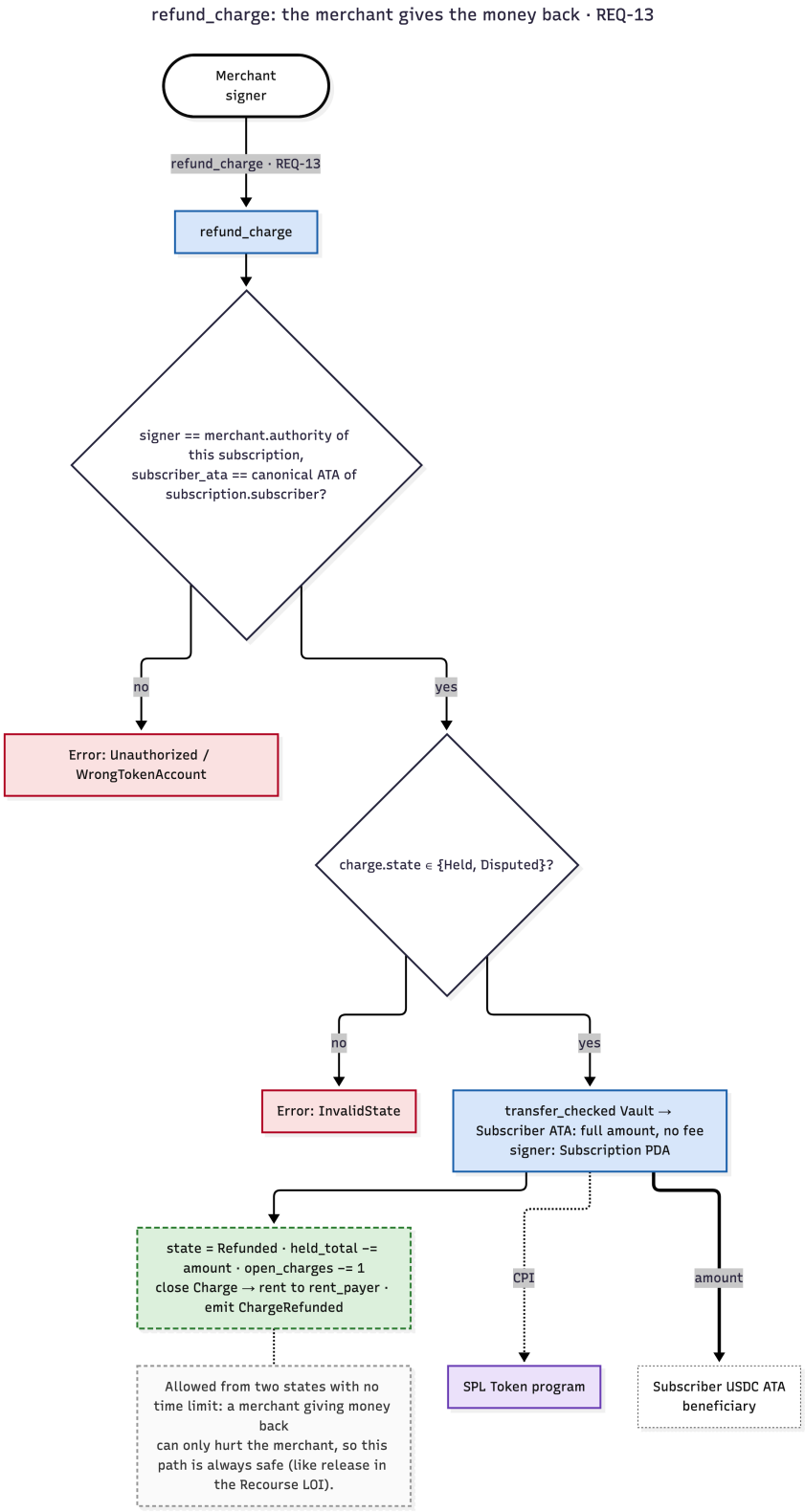


Figure 18: Instruction Flow: resolve_dispute [REQ-14]

The resolver named in the plan sets the split. Every destination is a stored address.

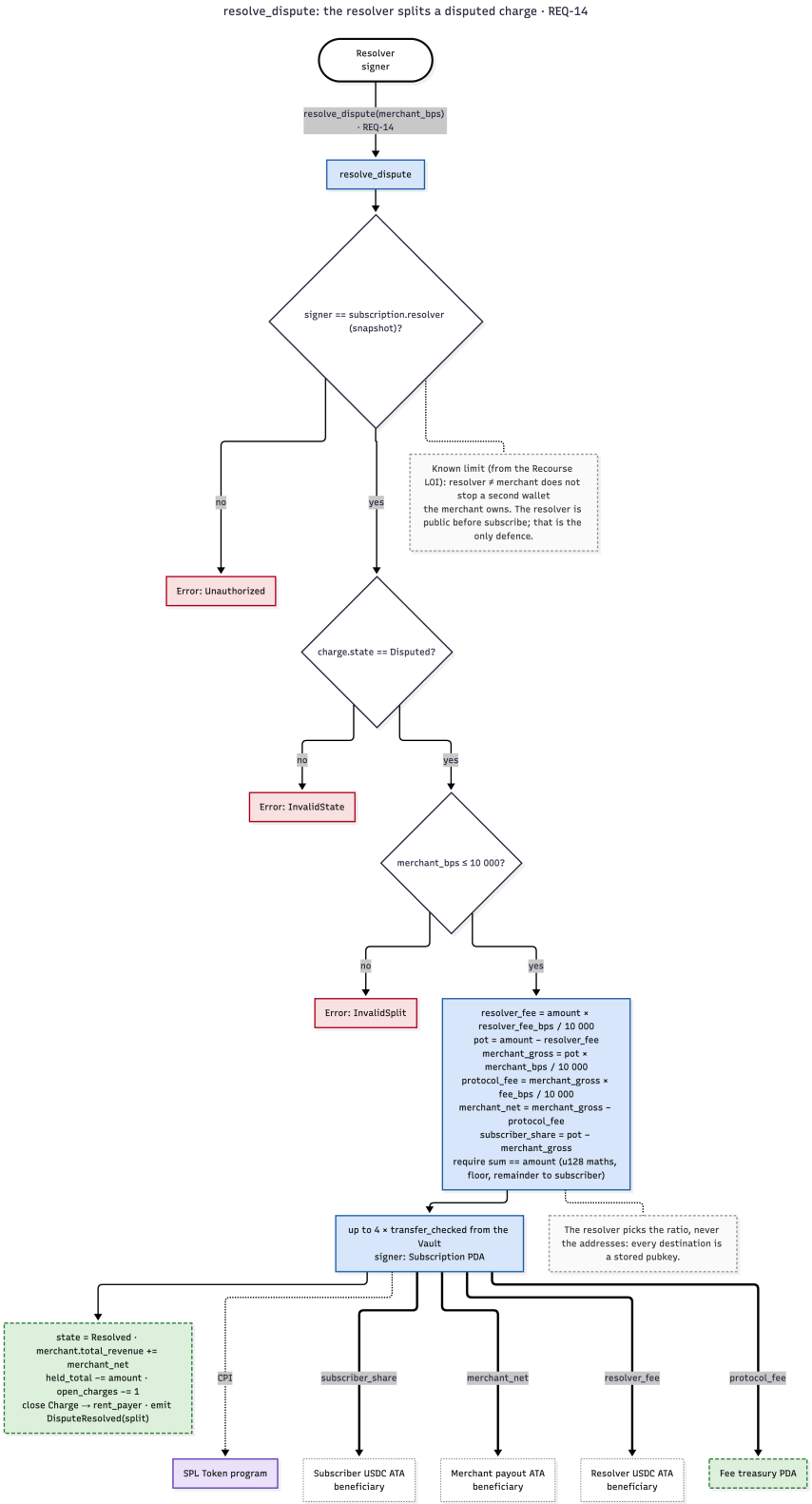
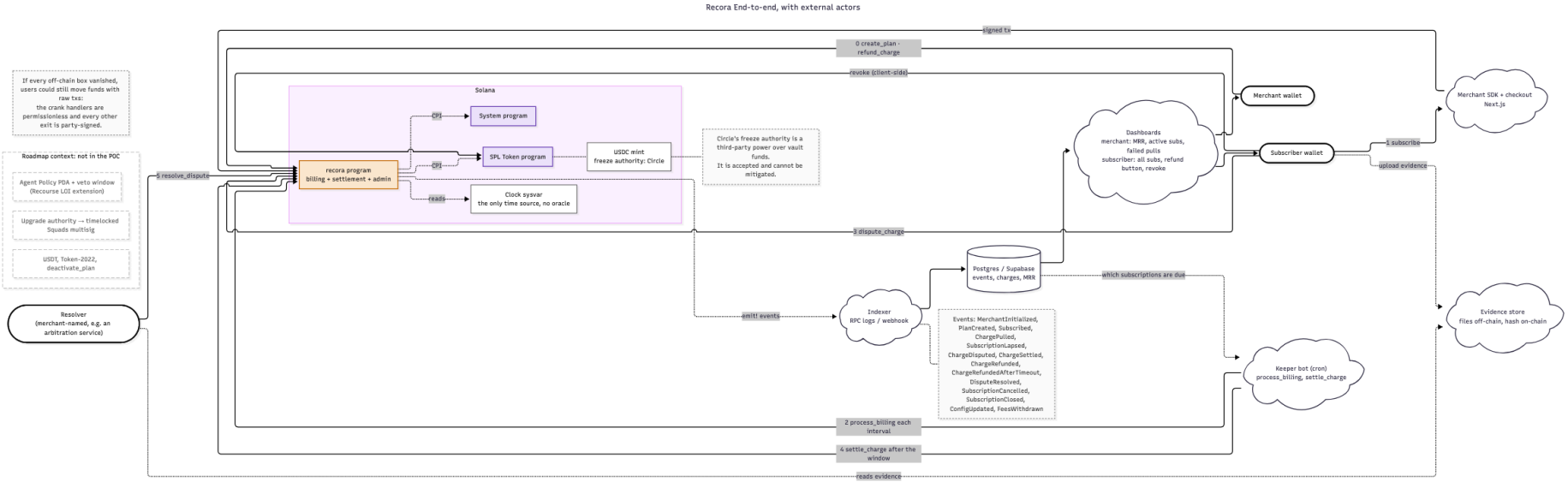


Figure 19: End-to-End View with External Actors

The off-chain services around the program (SDK, keeper, indexer, database, evidence store), the programs it calls, and roadmap items that are outside the POC.



5. Traceability Matrix

Requirement	Functional flow	Figure	Diagram label	Program handler
REQ-01	Initialize configuration	Figure 6	REQ-01 on the deployer arrow into initialize_config	`recora::initialize_config`
REQ-02	Update configuration	Figure 7	Figure 7 as a whole; one branch per admin instruction	`recora::update_config`
REQ-03	Withdraw fees	Figure 8	REQ-03 on the admin arrow into withdraw_fees	`recora::withdraw_fees`
REQ-04	Initialize merchant	Figure 9	REQ-04 on the merchant arrow into initialize_merchant	`recora::initialize_merchant`
REQ-05	Create plan	Figure 10	REQ-05 on the merchant arrow into create_plan	`recora::create_plan`
REQ-06	Subscribe (accounts only)	Figure 11	REQ-06 on the subscriber arrow into subscribe	`recora::subscribe`
REQ-07	Approve delegate	Figure 11	REQ-07 on both approve boxes	`recora::subscribe`
REQ-08	Pull into escrow	Figure 12	REQ-08 on the "yes" branch of the funding check	`recora::process_billing`
REQ-09	Transition to Lapsed	Figure 12	REQ-09 on the "no" branch of the funding check	`recora::process_billing`
REQ-10	Dispute charge	Figure 15	REQ-10 on the subscriber arrow into dispute_charge	`recora::dispute_charge`
REQ-11	Settle charge	Figure 16	REQ-11 on the "yes" branch of the review-window check	`recora::settle_charge`
REQ-12	Refund on dispute timeout	Figure 16	REQ-12 on the "yes" branch of the dispute-timeout check	`recora::settle_charge`
REQ-13	Merchant refund	Figure 17	REQ-13 on the merchant arrow into refund_charge	`recora::refund_charge`
REQ-14	Resolve dispute	Figure 18	REQ-14 on the resolver arrow into resolve_dispute	`recora::resolve_dispute`
REQ-15	Cancel subscription	Figure 13	REQ-15 on the subscriber arrow into cancel_subscription	`recora::cancel_subscription`
REQ-16	Close subscription	Figure 14	REQ-16 on the subscriber arrow into close_subscription	`recora::close_subscription`

The 16 requirements are carried by 13 instruction handlers. Three handlers carry two requirements each because they have two branches: `subscribe`, `process_billing` and `settle_charge`.